Java 入门笔记

第十二课 · 对象的创建、内存模型、this 与访问权限

学习目标	上节课学会了定义类、创建对象。本课先复习类与对象的关系，再看 `new` 背后发生了什么：对象在栈和堆里怎么存放，`this` 指向谁，以及如何用访问权限把数据“藏起来”。课堂代码是 `Car` 类，最后改成了带 `private` 字段的版本。

目录

本课目标 2

一、复习：类与对象 2

二、5.3 对象 Object 3

三、this 关键字 4

四、访问权限 5

五、课堂代码整理 6

常见错误 7

本课小结 8

自测题 8

整理日期：2026 年 11 月 3 日

本课目标

- 能说出类与对象的关系，以及面向对象的三大特征。
- 掌握对象创建的两步：声明引用、实例化。
- 能画出对象在栈、堆、方法区中的内存图。
- 会用 `this` 区分成员变量和同名参数。
- 记住四种访问权限的可见范围，会用 `private` 加 getter/setter 实现封装。

一、复习：类与对象

1. 类与对象的关系

类是对象的模板（抽象），对象是类的实例（具体）。 一个类可以创建多个对象，每个对象有自己的一份属性值，彼此互不影响。

概念	类比	例子
类	图纸、模具	`Car`：有颜色、速度，会飞
对象	按图纸造出的房子	红色、时速 180 的那辆车

老师的说法是“模板不占内存空间，也不参与实体的操作”。考试按这个记：类不存放对象的数据，只有 `new` 出来的对象才在堆里占空间。

更准确地说，类加载后，类的结构、方法代码和 `static` 变量都存放在方法区，所以类本身也占内存；`static` 方法也可以不创建对象、直接用类名调用，例如 `Math.max(3, 5)`。

2. 面向对象的三大特征

特征	含义	关键字 / 写法
封装	隐藏内部细节，只对外提供必要的接口	`private` 字段 + getter/setter
继承	子类复用父类的属性和方法，并可以扩展	`extends`，Java 只支持单继承
多态	同一个方法调用，不同对象表现不同	继承 + 重写 + 父类引用指向子类对象

记忆：封装藏起来，继承拿过来，多态变出来。 有些教材会再加上“抽象”，说成四大特征。

3. 类的定义与组成

```
[修饰符] class 类名 {
    // 成员变量（属性）：有什么
    // 构造方法：怎么造
    // 成员方法（行为）：能做什么
}
```

组成	作用	要点
成员变量	保存对象的状态	写在类中、方法外；有默认值（数值 0、`boolean` 为 `false`、引用为 `null`）
构造方法	`new` 时初始化对象	与类同名，没有返回值类型；不写时编译器补一个无参构造

组成	作用	要点
成员方法	描述对象的行为	`修饰符 返回值类型 方法名(参数) { }`

成员变量有默认值，局部变量没有默认值，必须先赋值才能使用。类名用大驼峰；一个 `.java` 文件只能有一个 `public` 类，且文件名必须和它同名。

4. 方法重载：两同一不同

- 两同：同一个类、方法名相同。
- 一不同：参数列表不同，即参数的个数、类型或顺序至少有一项不同。

只有这里不同	能否构成重载
参数个数、类型或顺序	能
返回值类型	不能，编译报错
参数名	不能，视为同一个方法
访问修饰符	不能

调用时可以不接收返回值（例如直接写 `add(1, 2);`），编译器只能靠方法名和参数来区分，所以看参数，不看返回值。详见 Class 11 的方法重载一节。

二、5.3 对象 Object

对象是类的具体实例，是封装属性与行为的基本单元。

1. 对象的创建

```
ClassName objectName;              // ① 声明对象引用：在栈中开辟空间存放引用
objectName = new ClassName();      // ② 实例化对象：在堆中分配空间并调用构造方法
ClassName obj = new ClassName();   // ③ 合并写法，最常用
```

`new` 做了三件事：

1. 在堆中为对象分配内存；
2. 给成员变量赋默认值或显式初始值，再调用构造方法；
3. 返回对象的地址，赋给栈中的引用变量。

只声明不实例化就使用，会出错：

```
Car a;
a.color = "red";      // 编译报错：局部变量 a 未初始化

Car b = null;
b.color = "red";      // 编译通过，运行时抛出 NullPointerException
```

2. 对象的使用

用点运算符 `.` 访问对象的属性和方法：`对象.属性`、`对象.方法(参数)`。

```
car.color = "red";
car.fly("black");
```

3. 对象的内存模型

区域	存放内容	特点
栈 Stack	局部变量、对象引用（地址）、方法调用帧	方法调用时分配，结束后自动释放
堆 Heap	`new` 出来的对象及其成员变量	线程共享；没有引用指向的对象由 GC 回收
方法区 Method Area	类的信息、方法代码、`static` 变量	类加载时写入；JDK 8 起称为元空间

以课堂代码为例，执行完两次 `new` 和赋值之后：

```
栈（main 方法帧）
car  ————>  堆：Car 对象 #1   color = "red"    seat = 5
car1 ————>  堆：Car 对象 #2   color = "green"  seat = 5

方法区：Car 类信息（字段定义、fly() 的代码）
        两个对象共用同一份方法代码
```

调用 `car.fly("black")` 时，栈上压入一个 `fly` 方法帧，里面有参数 `color = "black"`，以及隐含的 `this`，指向 `car` 所指的对象 #1。方法执行完，这个帧就弹出。

要点：

1. 属性每个对象各有一份，存在堆中；方法只有一份，所有对象共用。
2. 引用变量存的是地址。`Car c2 = car;` 不会创建新对象，`c2` 和 `car` 指向同一个对象，通过 `c2` 修改属性，`car` 也能看到变化。
3. `car = null;` 之后，如果对象 #1 没有其他引用，就会被垃圾回收。

> 字符串常量池在 JDK 7 以后移到了堆里；JVM 还有程序计数器、本地方法栈等区域。这些课上一般不考，了解即可。

三、this 关键字

`this` 代表当前对象，即正在调用这个方法的那个对象。`car.fly(...)` 执行时，`this` 就是 `car`；`car1.fly(...)` 执行时，`this` 就是 `car1`。

1. 区分成员变量和同名参数（必考）

课堂代码中，`fly` 的参数名和成员变量同名：

```
String color;                          // 成员变量

void fly(String color) {               // 参数，和成员变量同名
    System.out.println(this.color + " car is flying in " + color + " cloud");
}
```

方法里直接写 `color`，指的是参数（局部变量优先）；写 `this.color`，才是成员变量。所以 `car.fly("black")` 输出：

```
red car is flying in black cloud
```

在构造方法或 setter 里最常见：

```
Car(String color) {
    this.color = color;    // 左边是成员变量，右边是参数
}
```

若写成 `color = color;`，两边都是参数，成员变量始终是 `null`。

2. 调用本类的其他构造方法

```
Car() {
    this("white", 0);      // 必须是构造方法的第一条语句
}
Car(String color, int speed) {
    this.color = color;
    this.speed = speed;
}
```

3. 注意

- `static` 方法里不能用 `this`，因为 `static` 方法属于类，调用时可能没有对象。`main` 里不能写 `this.color`。
- 没有同名冲突时，`this.` 可以省略。

四、访问权限

访问权限控制类、属性和方法在不同范围内的可见性，是实现封装的核心手段。四种权限从严到宽：`private` < 默认 < `protected` < `public`。

修饰符	同一个类	同一个包	不同包的子类	不同包的其他类
`private`	✓	✗	✗	✗
默认（不写）	✓	✓	✗	✗
`protected`	✓	✓	✓	✗
`public`	✓	✓	✓	✓

口诀：私有只在本类，默认只在本包，保护多加子类，公开到处都行。

修饰符	典型用途
`private`	成员变量，配合 getter/setter
默认	包内协作的类和成员，也叫包访问权限
`protected`	留给子类使用的成员（讲继承时展开）

修饰符	典型用途
`public`	对外提供的类和方法，`main` 方法

- 默认权限是什么都不写，不是写 `default` 关键字。
- 顶层类只能是 `public` 或默认，不能是 `private` 或 `protected`。

课堂演示：把 **speed** 改成 **private**

课堂代码最后把 `speed` 改成了 `private`，`Test` 中原来的赋值语句就编译不过了：

```
class Car {
    String color;            // 默认权限：同包的 Test 可以访问
    private int speed;       // 私有：只有 Car 内部可以访问
    int sear = 5;
    ...
}

car.color = "red";           // ✓
car.speed = 180;             // ✗ 编译错误：speed has private access in Car
```

`private` 字段要通过类自己提供的方法来读写，这就是封装。课上 `Car` 里已经写了一个空的 `speed(int speed)` 方法，补全后的写法见下一部分。

五、课堂代码整理

把课堂代码补全为可以运行的版本：`speed` 保持私有，通过 `setSpeed` 和 `getSpeed` 读写，并修正输出中缺少的空格。

```
package class12;

public class Test {
    public static void main(String[] args) {
        Car car = new Car();           // 声明引用 + 实例化
        Car car1 = new Car();

        car.color = "red";             // 默认权限：同包可以直接访问
        car.setSpeed(180);             // private 字段只能通过方法修改
        car.fly("black");

        car1.color = "green";
        car1.fly("yellow");

        System.out.println(car.getSpeed() + " / " + car1.getSpeed());
        System.out.println(car.seat + " / " + car1.seat);
    }
}

class Car {
    String color;                      // 默认权限
    private int speed;                 // 私有：只有 Car 自己能直接访问
    int seat = 5;                      // 显式初始化

    void fly(String color) {
        // this.color 是成员变量，color 是参数
```

```
        System.out.println(this.color + " car is flying in " + color + " cloud");
    }

    void setSpeed(int speed) {
        if (speed >= 0) {
            this.speed = speed;
        }
    }

    int getSpeed() {
        return speed;
    }
}
```

运行结果：

```
red car is flying in black cloud
green car is flying in yellow cloud
180 / 0
5 / 5
```

从结果可以看出：

- `car1` 没有设置速度，`speed` 是默认值 `0`；
- `seat` 写了 `= 5`，每个新对象一创建就是 5；
- 两个对象的属性互不影响。

课堂原代码中的 `sear` 应为 `seat`（座位数），整理版已改正。

常见错误

- 方法定义时没有参数，调用时却传了实参：`void fly()` 不能写成 `car.fly("black")`，要在定义里声明形参。
- 形参与成员变量同名时，忘了写 `this.`，导致赋值落在参数上。
- 在其他类中直接访问 `private` 成员。
- 只声明引用、没有 `new` 就访问属性。
- 以为 `Car c2 = car;` 复制了一个新对象。
- 在 `static` 方法（如 `main`）里使用 `this`。
- 把 `main` 写成 `static void main`：JDK 25 起允许这种写法，但旧版本的 JDK 无法识别为程序入口，考试按 `public static void main(String[] args)` 写。

本课小结

- 类是对象的模板，对象是类的实例；类不存放对象的数据。
- 创建对象分两步：栈中声明引用，堆中 `new` 实例并调用构造方法。
- 属性每个对象一份，存在堆中；方法共用一份，存在方法区。
- `this` 代表当前对象，最常用于区分成员变量和同名参数。
- 四种访问权限从严到宽为 `private`、默认、`protected`、`public`；`private` 加 getter/setter 是封装的标准写法。

自测题

1. 执行 `Car c = new Car();` 时，栈和堆中分别多了什么？
2. `Car c2 = c1;` 之后执行 `c2.color = "blue";`，`c1.color` 是什么？为什么？
3. 下面的构造方法有什么问题？`Car(String color) { color = color; }`
4. `private`、默认、`protected`、`public` 中，哪一个能被同一个包里的其他类访问，却不能被不同包的子类访问？
5. 为什么 `main` 方法里不能使用 `this`？
6. 课堂代码把 `speed` 改为 `private` 后，`Test` 中应该怎样给汽车设置速度？